

RAG Mimarileri ve Araştırması

Bu bölüm en yoğun kısımdı. Önce RAG'in tek bir kalıp olmadığını, probleme göre çok farklı şekillere girebildiğini öğrendim; sonra da RAG'in ortak teknik bileşenlerine baktım.

A. Naive RAG ve Advanced RAG

Standart (naive) RAG'in dört adımdan oluştuğunu gördüm:



Bu hattın nerede tıkanıldığını araştırınca: kullanıcı sorusu kötü ifade edilirse arama tutmuyor, getirilen parçalar alakasız/gürültülü olabiliyor, hepsi de eşit önemliymiş gibi modele veriliyor. Advanced RAG bunu iki katmanla düzeltiyormuş: arama öncesinde soruyu yeniden yazmak ya da önce varsayımsal bir cevap üretip ona göre arama yapmak (buna **HyDE** deniyor); arama sonrasında da bulunan parçaları yeniden sıralamak (re-ranking) ve gereksiz kısımları kırmak (bağlam sıkıştırma). Sonuçta modele giden bağlam hem daha isabetli hem daha az gürültülü oluyor.

Veri Yapısına Göre Değişen RAG'ler

Vector-based RAG'de her parça kendi başına bir vektör, aralarında bir bağ yok — sadece anlamca ne kadar yakın oldukları önemli. **GraphRAG** ise bunun bir adım ötesi: varlıklar (kişi, şirket vs.) ve aralarındaki ilişkiler bir bilgi grafiği olarak çıkarılıyor. Bunun avantajını şu örnekle daha iyi anladım: "X şirketinin CEO'sunun daha önce çalıştığı şirket hangi sektördeydi?" gibi çok adımlı bir soruda, vektör-only arama bu zinciri tek bir parçada bulamıyor ama graf ilişkileri takip edebildiği için bu tarz sorularda çok daha güçlü kalıyor.

Multimodal RAG'de ise iş metinle sınırlı kalmıyor — görseller, tablolar, hatta ses de işlenebiliyormuş; her biri için ayrı bir ön işleme adımı (görsel için embedding

ya da metne çevirme, ses için transkripsiyon gibi) gerekiyor.

Akış ve Mantık Odaklı RAG'ler

Corrective RAG (CRAG), bulduğu dokümanların kalitesini bir değerlendiriciyle puanlıyor, kalite düşükse ek bir arama tetikliyor. **Self-RAG** ise modelin kendi ürettiği cevabı kendi kendine denetlemesi — "bu iddiayı gerçekten bağlamdan mı çıkardım?" diye. Farkını şöyle anladım: CRAG bulduğun veriye bakıyor, Self-RAG ürettiğin cevaba bakıyor.

Agentic RAG'de ise sabit bir hat yok, kararları bir ajana bırakıyorsun — hangi kaynağa bakacağına, web'e çıkıp çıkmayacağına o karar veriyor. Avantajı esneklik, ama riski de var: öngörülemez olabiliyor, birden fazla model çağrısı yaptığı için daha yavaş/pahalı, ve hata ayıklaması da zorlaşıyor.

RAG ÇEŞİTLERİ – KENDİ ÖZETİM		
YAKLAŞIM	TEMEL FARK	NEREDE AVANTAJLI
Advanced RAG	Arama öncesi/sonrası ek katmanlar	Genel isabet artırma
GraphRAG	İlişkileri graf üzerinden takip eder	Çok adımlı, ilişkisel sorular
Multimodal RAG	Metin dışı veri tiplerini işler	Görsel/tablo/ses içeren kaynaklar
CRAG	Getirilen veriyi puanlayıp düzeltir	Retrieval kalitesi değişkense
Self-RAG	Modelin kendi cevabını denetler	Halüsinasyon riskini azaltma
Agentic RAG	Kararları bir ajana bırakır	Karmaşık, çok kaynaklı görevler

B. RAG'in Temel Teknik Bileşenleri

Chunking kısmında üç yöntem gördüm: sabit boyutlu bölme (basit ama cümle ortasından kesebiliyor), cümle tabanlı bölme (daha doğal ama parça boyutları eşit olmuyor), anlamsal bölme (embedding'deki ani değişime göre bölüyor, konuyu bölmüyor ama hesaplaması pahalı). **Örtüşme (overlap)** kavramını da öğrendim — parçalar arasında biraz tekrar bırakmak, bölme noktasında bilginin kesilip kaybolmasını önüyor.

PARÇA 1

...retrieval sistemleri kullanıcı sorusunu alır

ve en yakın parçaları bulur.

PARÇA 2

ve en yakın parçaları bulur.

Bu parçalar LLM'e bağlam olarak...

Benzerlik metrikleri tarafında: kosinüs benzerliği iki vektör arasındaki açığı ölçüyor, en yaygın kullanılanı bu çünkü metin uzunluğundan etkilenmiyor. İç çarpım hem açığı hem büyüklüğü hesaba katıyor. Öklid uzaklığı ise düz çizgi mesafesi — ama embedding'ler genelde yüzlerce boyutlu olduğu için burada ayırt ediciliğinin zayıfladığını okudum.

VEKTÖRLER ARASI BENZERLİK ÖLÇÜTLERİ

METRIK	NE ÖLÇER	NOT
Kosinüs Benzerliği	İki vektör arasındaki açı	Metin uzunluğundan bağımsız, en yaygın tercih
İç Çarpım	Açı + büyüklük (magnitude)	Normalize vektörlerde kosinüsle eşdeğer
Öklid Uzaklığı	Düz çizgi mesafesi	Yüksek boyutta ayırt ediciliği zayıflar

Halüsinasyonu engellemek için üç şey öne çıkıyor: istem içinde "sadece verilen bağlamdaki bilgiyi kullan, yoksa 'bilgi bulunamadı' de" gibi net bir kısıtlama koymak, modelin iddialarını hangi parçaya dayandırıldığını göstermesini istemek (kaynak gösterme), ve üretim sırasında düşük bir temperature kullanmak.